A VALID 14-POINT BLACKOUT OF THE 5×8 GRID: THE CONJECTURE $T(n, m) = n + m$ OF OEIS A397315 FAILS AT $(5, 8)$

MATH0

Abstract. Let $T(n, k)$ (OEIS A397315) be the largest number of points of an $n \times k$ grid of points that can be blacked out so that every rectangle with all four vertices on grid points, tilted ones included, is determined by its set of visible vertices. A. Pemmasani conjectured in 2026 that $T(n, m) = n + m$ whenever $\min(n, m) \geq 4$, apart from $T(4, 4) = 7$ and $T(4, 5) = 8$. We exhibit 14 points of the 5×8 grid whose removal leaves the 364 rectangles of the grid with pairwise distinct sets of visible vertices, so $T(5, 8) \geq 14 > 13$ and the conjecture is false. The result is known: on October 17, 2026, Pemmasani recorded $T(5, 8) = 14$ in the repository linked from the entry, and on October 18, 2026 a Lean proof of $T(5, 8) = 14$ appeared there; this note is an independent formal proof of a known result. Our refutation is verified in Lean 4 against a frozen, fidelity-reviewed statement that math0 formalized from the OEIS entry, whose definitions are checked by the kernel against the first ten listed terms.

1. Introduction

Problem 001 of Peter Kagey's Open Problem Collection [2] asks how many points of a rectangular grid of points can be blacked out while every rectangle whose four vertices are grid points remains identifiable from its visible vertices. The OEIS entry A397315, contributed by Arjun Pemmasani on September 2, 2026 [5], tabulates the answer $T(n, k)$ for the $n \times k$ grid, $n, k \geq 2$, by antidiagonals; the companion entry A399580 [6], by the same author, counts the maximum blackouts. The 45 listed terms cover the antidiagonals $n + k \leq 12$. The comments of A397315 state $T(2, m) = m + 1$ for $m \geq 3$, with reference to the linked repository, and two conjectures: $T(3, m) = m + 2$ for $m \geq 3$, and the one quoted in Section 2, that $T(n, m) = n + m$ whenever $\min(n, m) \geq 4$, with two exceptions. All listed cells with $\min(n, k) \geq 4$ agree with it.

Result. The second conjecture fails on the next antidiagonal: the 5×8 grid has a valid blackout of 14 points (Theorem 2, Figure 1), so $T(5, 8) \geq 14$, whereas the conjecture predicts $T(5, 8) = 13$.

Prior work and status. The entry links Pemmasani's GitHub repository [7], which, as the math0 referees found, already contains the refutation. The ledger `kagey-problems/problem001/data/ledger.md`, section "Outcomes, part 2 (2026-10-17)" (commit `45b175b`, October 17, 2026), records `blackout(5,8) = 14` and `blackout(5,9) = 15`, against the predicted 13 and 14, also `blackout(6,8) = 15`, and that the maximum blackouts of the 5×8 grid form one orbit of size 4 under the symmetries of the grid. The paper draft `kagey-problems/problem001/jis/pemmasani.tex` states, with f for T, $f(5, 8) = 14$ and $f(6, 8) = 15$ (theorem "the guess fails") and $f(5, m) = m + 6$ for every $m \geq 8$ (theorem "five rows"). The Lean file `kagey-problems/lean/KageyProblems/Problem001/Kernel.lean` (commit `8df4162`, October 18, 2026) contains a kernel-checked theorem `kfvalue_5_8` stating $f(5, 8) = 14$, and `CollarBoards.lean` (added in commit `a46acfd`, later on October 18, 2026) contains `fval_five`, $f(5, m) = m + 6$ for $m \geq 8$. The referees also observed that our blackout is the image of Pemmasani's maximum blackout S_8 under the reflection $y \mapsto 4 - y$. At the time of writing, neither OEIS entry (A397315, revision #38 of September 12, 2026; A399580,

Date: October 18, 2026.

revision #24) has been updated with the refutation, and A397315 still states the conjecture. The conjecture is not among the statements of the formal-conjectures repository [1]. Our witness was found by a SAT search in the math0 pipeline, and our Lean proof was written without use of Pemmasani's files. Since a Lean proof of the stronger statement $T(5,8) = 14$ already exists, we classify this note as an *independent formal proof of a known result.* We prove only $T(5,8) \geq 14$, which suffices for the refutation.

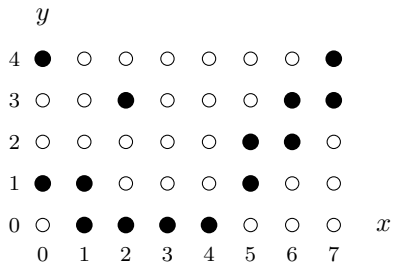


FIGURE 1. The blackout B^* (filled) in $G_{5,8} = \{0,\dots,7\} \times \{0,\dots,4\}$.

2. Statement

Definitions. Following the programs in the entry, the $n \times k$ grid is $G_{n,k} = \{0,\dots,k-1\} \times \{0,\dots,n-1\} \subseteq \mathbb{Z}^2$. A *rectangle* of $G_{n,k}$ is a set $\{a,b,c,d\}$ where $a,b,c \in G_{n,k}$ are pairwise distinct, $(a-b)\cdot(c-b) = 0$, and $d = a+c-b \in G_{n,k}$; let $\mathcal{R}_{n,k}$ be the set of rectangles. A set $B \subseteq G_{n,k}$ is a *valid blackout* if $R \mapsto R \setminus B$ is injective on $\mathcal{R}_{n,k}$, and $T(n,k)$ is the largest size of a valid blackout (the empty set is valid). Here $d \notin \{a,b,c\}$: $d = a$ forces $c = b$, $d = c$ forces $a = b$, and $d = b$ gives $0 = (a-b)\cdot(c-b) = -|a-b|^2$. So a,b,c,d are the vertices, in cyclic order, of a non-degenerate rectangle with a right angle at b, and conversely every rectangle with vertices in $G_{n,k}$ (axis-parallel or tilted, squares included) arises in this way.

The conjecture. The conjecture is stated in the COMMENTS section of A397315 (entry by A. Pemmasani, September 2, 2026; revision #38 of September 12, 2026), verbatim:

> `Conjecture: T(n,m) = n+m whenever min(n,m) >= 4, except T(4,4) = 7 and T(4,5) = 8 (verified on seven cells through 6 X 6).`

Since T is symmetric (transpose the grid) and the entry lists $T(5,4) = 8$, the exception $T(4,5) = 8$ is read as covering $(5,4)$ as well.

Conjecture 1 (Pemmasani [5])**.** *For all n,m with $\min(n,m) \geq 4$: $T(4,4) = 7$, $T(4,5) = T(5,4) = 8$, and $T(n,m) = n+m$ otherwise.*

The cell $(5,8)$ is not exceptional, so under every reading of the exceptions the conjecture predicts $T(5,8) = 13$.

Theorem 2. *The set*

$$B^* = \{(0,1),(0,4),(1,0),(1,1),(2,0),(2,3),(3,0),\\ (4,0),(5,1),(5,2),(6,2),(6,3),(7,3),(7,4)\}$$

is a valid blackout of $G_{5,8}$. Hence $T(5,8) \geq 14 > 5+8$, and Conjecture 1 is false.

Lean formalization. The statement was formalized by math0 in `lean/Math0/Statements/OEIS/A397315.lean`, reviewed for fidelity, and frozen (SHA-256 `81afd722...39eba61c5d`) before any proof attempt. The definitions on which `statement` depends are, verbatim except that docstrings and blank lines are omitted:

```
namespace Math0.Statements.A397315
def pt (n i : ℕ) : ℤ × ℤ := (((i / n : ℕ) : ℤ), ((i % n : ℕ) : ℤ))
def inGrid (n k : ℕ) (p : ℤ × ℤ) : Bool :=
  decide (0 ≤ p.1) && decide (p.1 < k) && decide (0 ≤ p.2) && decide (p.2 < n)
def idx (n : ℕ) (p : ℤ × ℤ) : ℕ := p.1.toNat * n + p.2.toNat
def rectMask? (n k a b c : ℕ) : Option ℕ :=
  let A := pt n a
  let B := pt n b
  let C := pt n c
  let D := A + C - B
  if a ≠ b ∧ b ≠ c ∧ a ≠ c ∧ (A.1 - B.1) * (C.1 - B.1) + (A.2 - B.2) * (C.2 - B.2) = 0 ∧
      inGrid n k D = true then
    some (2 ^ a ||| 2 ^ b ||| 2 ^ c ||| 2 ^ idx n D)
  else none
def rects (n k : ℕ) : List ℕ :=
  ((List.range (n * k)).flatMap fun a => (List.range (n * k)).flatMap fun b =>
    (List.range (n * k)).filterMap fun c => rectMask? n k a b c).eraseDups
def visible (R B : ℕ) : ℕ := Nat.xor R (Nat.land R B)
def notMem (x : ℕ) (l : List ℕ) : Bool := l.rec true fun y _ ih => !(Nat.beq x y) && ih
def pairwiseDistinct (l : List ℕ) : Bool := l.rec true fun x xs ih => notMem x xs && ih
def isValidBlackout (n k B : ℕ) : Bool := pairwiseDistinct ((rects n k).map (visible · B))
def card : ℕ → ℕ → ℕ
  | 0, _ => 0
  | len + 1, B => B % 2 + card len (B / 2)
def T (n k : ℕ) : ℕ :=
  ((List.range (2 ^ (n * k))).filter (isValidBlackout n k)).foldr
    (fun B m => max (card (n * k) B) m) 0
def statement : Prop :=
  ∀ n m : ℕ, 4 ≤ min n m →
    T n m = if n = 4 ∧ m = 4 then 7
      else if (n = 4 ∧ m = 5) ∨ (n = 5 ∧ m = 4) then 8
      else n + m
end Math0.Statements.A397315
```

Encoding. The point $(x, y) \in G_{n,k}$ has index $nx+y \in \{0, \dots, nk-1\}$, and `pt n i` $= (\lfloor i/n \rfloor, i \bmod n)$ inverts this numbering; a set S of points is the bitmask $\sum_{(x,y)\in S} 2^{nx+y}$. For indices a, b, c, `rectMask?` tests the conditions of the definition, with the dot product computed in $\mathbb{Z}$, and returns the mask of $\{a, b, c, d\}$. The list `rects n k` collects these masks over all ordered triples and erases duplicates (each rectangle arises from exactly 8 triples: b is any vertex and a, c are its two neighbours, in either order), so it lists $\mathcal{R}_{n,k}$ once each. `visible R B` $= R \oplus (R \wedge B)$ is the mask of $R \setminus B$, and `pairwiseDistinct` decides `List.Nodup` (Lemma 6), so `isValidBlackout n k B` says that B is a valid blackout. `card (n*k) B` counts the 1 bits of B below position nk, and `T n k` maximizes it over the valid masks $B < 2^{nk}$; this is $T(n, k)$, and `statement` is Conjecture 1.

Check against the OEIS data. The statement file also defines the antidiagonal reading order `cell` and the sequence `a` with the entry's offset 2. Ten theorems `T_2_2`, …, `T_5_2`, each proved by `decide +kernel`, evaluate T from the definition on the first ten cells (at most 2^{12} candidate blackouts each), and `a_data` assembles them: the terms $4, 4, 4, 5, 5, 5, 6, 6, 6, 6$ agree with the entry. During formalization and fidelity review, Python transcriptions of the same encoding reproduced the first 15 listed terms (among them $T(4, 4) = 7$) and $T(4, 5) = T(5, 4) = 8$, and

a referee's exact MaxSAT computation reproduced 13 listed values, among them $T(5,5) = 10$, $T(6,6) = 12$ and $T(5,7) = 12$.

3. Proof

The proof follows `lean/Math0/Frontier/OEIS/A397315.lean`; names of private auxiliary declarations are given in brackets. Throughout, $n = 5$, $k = 8$, and points are identified with their indices $0, \dots, 39$. In indices, the blackout and its mask [`Bw`] are

$$B^* = \{1,4,5,6,10,13,15,20,26,27,32,33,38,39\}, \qquad B_w = \textstyle\sum_{i\in B^*} 2^i = 837721039986 < 2^{40}.$$

Evaluating `isValidBlackout 5 8 Bw` directly in the kernel exhausts memory (evaluating `rectMask?` alone on the 40^3 triples took about 219 s and 3.9 GB). The proof therefore replaces `rectMask?` by a natural-number version and splits the check into a covering step and an injectivity step.

Lemma 3 (`rectMask_eq`). *For $i \in \mathbb{N}$ write $i_x = \lfloor i/n \rfloor$, $i_y = i \bmod n$, and let $\dot{-}$ be truncated subtraction. For all $n, k, a, b, c \in \mathbb{N}$, `rectMask? n k a b c` equals `fastMask n k a b c`, defined as `some`$(2^a \mid 2^b \mid 2^c \mid 2^\delta)$, $\delta = (a_x + c_x \dot{-} b_x)\, n + (a_y + c_y \dot{-} b_y)$, if a, b, c are pairwise distinct and*

$$a_x c_x + b_x^2 + a_y c_y + b_y^2 = a_x b_x + b_x c_x + a_y b_y + b_y c_y, \tag{1}$$

$$b_x \le a_x + c_x < k + b_x, \qquad b_y \le a_y + c_y < n + b_y, \tag{2}$$

and `none` otherwise ($\mid$ is bitwise or).

Proof. In `rectMask?`, $A = (a_x, a_y)$, $B = (b_x, b_y)$, $C = (c_x, c_y) \in \mathbb{Z}^2$. The expression $(a_x - b_x)(c_x - b_x) + (a_y - b_y)(c_y - b_y)$ is the left side of (1) minus its right side, so it vanishes in $\mathbb{Z}$ if and only if (1) holds in $\mathbb{N}$ (`linear_combination` and injectivity of $\mathbb{N} \to \mathbb{Z}$). The point $D = (a_x + c_x - b_x,\ a_y + c_y - b_y)$ is in the grid if and only if (2) holds, and then its coordinates are nonnegative, so `toNat` agrees with truncated subtraction and `idx n D` $= \delta$ (`omega`). □

Let L_{r} [`Lr`] be the explicit sorted list of 364 numbers in the proof file, from $99 = 2^0 + 2^1 + 2^5 + 2^6$ (the unit square at the origin) to 850403524608.

Lemma 4 (`chk_0`–`chk_39`, `F_true`, `mem_rects`). *Every element of `rects 5 8` belongs to L_{r}.*

Proof. Let $F(a,b,c)$ [`F`] be `true` if `fastMask 5 8 a b c` is `none`, and otherwise, if it is `some r`, the result of a linear scan of L_{r} for r by `Nat.beq` [`scan`, `memL`]. Let $\mathrm{chkA}(a) = \bigwedge_{b<40} \bigwedge_{c<40} F(a,b,c)$, written as two nested `Nat.rec` loops [`chkA`]. For each $a < 40$, the theorem `chk_a : chkA a = true` is proved by `decide +kernel`: the kernel evaluates $F(a,b,c)$ for the 1600 pairs (b,c) and reduces $\mathrm{chkA}(a)$ to `true`. Altogether this covers the 64000 triples, of which $2912 = 8 \cdot 364$ yield a mask and trigger a scan of L_{r}. By induction on m, if the loop `Nat.rec true (fun c ih => f c && ih) m` evaluates to `true`, then $f(c) =$ `true` for all $c < m$ [`loop_spec`]; applied twice, with `interval_cases` on a [`chk_all`], this gives $F(a,b,c) =$ `true` for all $a, b, c < 40$. A successful scan implies membership, by induction on the list [`mem_of_scan`]. Now let $r \in$ `rects 5 8`. Every element of l`.eraseDups` lies in l, so `rectMask? 5 8 a b c = some r` for some $a, b, c < 40$. By Lemma 3, `fastMask 5 8 a b c = some r`, and $F(a,b,c) =$ `true` says that $r \in L_{\mathrm{r}}$. □

Only the inclusion is used, but L_{r} is exactly the set of rectangle masks of $G_{5,8}$ (Python computations by math0 and a referee, who also obtained 364 rectangles from the entry's own program); $280 = \binom{8}{2}\binom{5}{2}$ of them are axis-parallel and 84 are tilted.

Lemma 5 (`dist`). *`pairwiseDistinct (Lr.map (visible · Bw)) = true`: the 364 masks of $R \setminus B^*$, $R \in L_{\mathrm{r}}$, are pairwise distinct.*

Proof. A closed Boolean computation, proved by `decide +kernel`: the kernel computes the masks $R \oplus (R \wedge B_w)$ and makes the $\binom{364}{2} = 66066$ comparisons by `Nat.beq`, all returning `false`. □

Lemma 6 (`pd_iff`, `nodup_eraseDups'`)**.** *For lists l of natural numbers, `pairwiseDistinct` l = `true` if and only if l has no duplicates, and l.`eraseDups` has no duplicates.*

Proof. The first claim follows by induction on l from `notMem` $x\ l$ = `true` $\iff$ $x \notin l$ [`notMem_iff`], itself proved by induction using `Nat.beq` $x\ y$ = `true` $\iff$ $x = y$. For the second, use strong induction on the length: $(x :: xs)$.`eraseDups` $= x :: (xs$.`filter`$(\cdot \neq x))$.`eraseDups`, the head does not occur in the tail, and the filtered list is shorter. □

Proposition 7 (`valid`)**.** *`isValidBlackout 5 8 Bw = true`; that is, B^* is a valid blackout of $G_{5,8}$.*

Proof. By Lemma 6 we must show that `(rects 5 8).map (visible · Bw)` has no duplicates. The list `rects 5 8` has none (Lemma 6). By Lemmas 5 and 6, $R \mapsto$ `visible`(R, B_w) is injective on the elements of L_{r}, hence, by Lemma 4, on those of `rects 5 8`; the image of a duplicate-free list under a map injective on its elements is duplicate-free (`List.Nodup.map_on`). □

Proof of Theorem 2. The kernel checks `card (5 * 8) Bw = 14` by `decide +kernel` [`card_Bw`], and $B_w < 2^{40}$ holds by `norm_num`. With Proposition 7, B_w belongs to the list (`List.range` 2^{40}).`filter`(`isValidBlackout 5 8`) whose `foldr` of `max` defines `T 5 8`; such a fold is at least its value at each element [`le_foldr`], so $14 \le$ `T 5 8` [`T_5_8_ge`]. Assume `statement`. Since $\min(5, 8) \ge 4$ and neither exception applies, it gives `T 5 8` $= 5 + 8 = 13$, a contradiction:

```
theorem result : ¬ statement := by
  intro h
  have h58 := h 5 8 (by norm_num)
  have h14 := T_5_8_ge
  rw [h58] at h14
  norm_num at h14
```

□

A check with fewer comparisons (not formalized). Any three vertices of a rectangle are a vertex v and its two neighbours u, w, forming a right triangle with its unique right angle at v; so they determine the rectangle (the fourth vertex is $u + w - v$). Hence two distinct rectangles can have the same visible set only if it has at most two points, and B is valid if and only if at most one rectangle lies in B, no two rectangles with exactly three vertices in B share their visible vertex, and no two with exactly two vertices in B share their visible pair. For B^* the numbers of rectangles with $0, 1, 2, 3, 4$ vertices in B^* are $53, 148, 139, 24, 0$, and the 24 visible points and 139 visible pairs are pairwise distinct (Python computation). Adding any of the 26 points of $G_{5,8} \setminus B^*$ to B^* gives an invalid blackout, so B^* is maximal under inclusion.

Remark 8 (Further values; not formalized)**.** The math0 SAT computations (CaDiCaL) and a referee's MaxSAT computation give $T(5, 8) = 14$ exactly, in agreement with Pemmasani's `kfvalue_5_8` [7]. The SAT computations also found valid blackouts showing $T(5, m) \ge m + 6$ for $9 \le m \le 12$, consistent with Pemmasani's $f(5, m) = m + 6$, and report $T(4, m) = m + 4$ for $9 \le m \le 11$, $T(6, 7) = 13$, and $T(3, m) = m + 2$ for $3 \le m \le 25$.

4. Machine verification

Files and declarations. The proof is in `lean/Math0/Frontier/OEIS/A397315.lean` of the math0 repository, `https://github.com/Foundation-0/math0`. The main theorem is `Math0.Frontier.A397315.result`, of type `¬ Math0.Statements.A397315.statement`. The proof file contains no `sorry`, `admit`, `native_decide`, `axiom` or `set_option` (the statement file sets `maxRecDepth` for four data checks).

Kernel computation. The finite computations are the 40 checks `chk_0`, ..., `chk_39`, `dist` and `card_Bw`, all by `decide +kernel`: the kernel reduces the Boolean to `true` while type-checking the proof term, using its built-in arithmetic on natural-number literals. No compiled code is trusted, unlike with `native_decide`, which would add the axiom `Lean.ofReduceBool`. The file compiles in about 60 to 95 s with peak memory about 2.5 GB.

Audit. The file ends with `#math0_settles Math0.Frontier.A397315.result Math0.Statements.A397315.statement`. This command (`lean/Math0/Audit.lean`) fails unless (1) `statement` is a definition of type `Prop` without universe parameters; (2) `statement` and `result` depend on no axioms besides `propext`, `Classical.choice` and `Quot.sound`, which rejects `sorryAx`, the axioms `Lean.ofReduceBool` and `Lean.trustCompiler` of `native_decide`, and custom axioms; and (3) the type of `result` is syntactically the statement or its negation, up to metadata and with no unfolding. Running `lake env lean` on the file (in `lean`) reports

```
math0 audit: 'Math0.Statements.A397315.statement' depends on axioms [propext]
math0 audit: 'Math0.Frontier.A397315.result' depends on axioms
  [Classical.choice, Quot.sound, propext]
math0 statement check: 'Math0.Frontier.A397315.result' refutes
  'Math0.Statements.A397315.statement'
```

Toolchain and reproduction. We use Lean 4 [4] `v4.33.1` and Mathlib [3] `v4.33.1`, pinned in `lean/`; the repository also pins formal-conjectures [1] at commit `c77fdd4`, which this result does not use. To reproduce, run `scripts/verify.sh` from the repository root: it builds all math0 results with `lake build`, re-running every audit command, and checks that the pinned dependencies are unmodified.

References

[1] M. Firsching, P. Lezeau, S. Mercuri, M. Z. Horváth, Y. Dillies, C. Sönne, E. Wieser, F. Zhang, T. Hubert, B. Agüera y Arcas, and P. Kohli, *Formal Conjectures: An open and evolving benchmark for verified discovery in mathematics*, arXiv:2605.13171, 2026. Repository: `https://github.com/google-deepmind/formal-conjectures`.

[2] P. Kagey, *Open Problem Collection, Problem 001*, `https://peterkagey.com/problems/001/`.

[3] The mathlib Community, *The Lean mathematical library*, in: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), ACM, 2020, 367–381.

[4] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, in: Automated Deduction – CADE 28, Lecture Notes in Comput. Sci. **12699**, Springer, 2021, 625–635.

[5] The OEIS Foundation Inc., `https://oeis.org/A397315`, entry by A. Pemmasani (2026).

[6] The OEIS Foundation Inc., `https://oeis.org/A399580`, entry by A. Pemmasani (2026).

[7] A. Pemmasani, *Solvers, exhaustive verification, and pre-registration ledger for Problem 001*, GitHub repository `https://github.com/apemm/Kagey-Problems` (commits `45b175b`, October 17, 2026, and `8df4162` and `a46acfd`, October 18, 2026).